Session 4.3: AI Applications in Business Decision-Making Part 2: Pricing Optimization and Revenue Management

TODAY'S PURPOSE

Why This Session Matters

- Set prices with data instead of guesswork
- Balance revenue, margin, and inventory
- Plan markdowns that clear stock on time

Today's flow: 60 min lecture, then 120 min hands-on lab

EXPECTED OUTCOMES

By the end of today, you will be able to:

1. Calculate the Price Elasticity of Demand (PED)
2. Simulate revenue and margin across prices
3. Recommend markdowns that clear inventory on time

Concepts for This Session

CONCEPT 1

Price Elasticity of Demand (PED)

The percentage change in quantity demanded resulting from a 1% change in product price, quantifying consumer price sensitivity.

CONCEPT 2

Revenue Management Optimization

Programmatically identifying the price point that maximizes total gross profit ((Price - Unit_Cost) Quantity) subject to market demand curves.

CONCEPT 3

Gross Margin Simulation

Simulating projected sales volume and financial profit across discrete pricing scenarios (+5%, +10%, -10%).

CONCEPT 4

Competitor Price Gap Sensitivity

Modeling how the premium or discount relative to competitor catalog prices influences customer purchase conversion.

How the Concept Works in Practice

FOUNDATION 1

Microeconomic Demand Curves in Empirical Machine Learning

Translating economic theory into executable code. Rather than assuming static demand curves, machine learning regression estimates empirical price sensitivity directly from historical sales and competitor price variations.

FOUNDATION 2

Revenue Optimization vs. Margin Maximization

Maximizing total revenue does not always maximize operating profit. Lowering prices may increase unit sales while eroding gross margins. Algorithms compute the exact price that maximizes bottom-line profit dollars.

FOUNDATION 3

Dynamic Response to Competitor Shifts

Consumer e-commerce purchase decisions depend heavily on relative price differences. Models incorporate the ratio of Current_Price / Competitor_Price as a primary explanatory feature.

FOUNDATION 4

Managing Inventory Expiration and Velocity

High inventory holding costs and obsolescence risks require dynamic price adjustments. If daily sales velocity falls behind targets, automated algorithms apply markdown tiers to clear inventory before fiscal year-end.

Build, Test, and Diagnose in the IDE

[1] FILE EXPLORER

Organize Assets

Use the session dataset and keep scripts in a clear workspace.

[2] CODE EDITOR

Write Small Segments

Run each completed segment before expanding the program.

[3] AI AGENT CHAT

Diagnose Errors

Capture the traceback, request an explanation, and review the fix.

[4] INTEGRATED TERMINAL

Verify Execution

Confirm the final program completes with return code 0.

Script Demonstration 1: AI Applications in Business Decision-Making Part 2: Pricing Optimization and Revenue Management

```
import pandas as pd
    import numpy as np

    df = pd.read_csv("data/session_04_03_dataset.csv")

    # Percentage premium relative to competitor
    df["Price_Premium_Pct"] = ((df["Current_Price_USD"] -
df["Competitor_Price_USD"]) / df["Competitor_Price_USD"]) * 100.0

    # Empirical Price Elasticity: % Change in Sales / % Change in
Price
    base_price = df["Current_Price_USD"].mean()
    base_sales = df["Historical_Daily_Sales_Units"].mean()

    # Simulated elasticity: For every 10% price premium, daily sales
drop by 15% (Elasticity = -1.5)
    elasticity_coefficient = -1.5
    print(f"Empirical Price Elasticity of Demand Coefficient:
{elasticity_coefficient}")
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 2: AI Applications in Business Decision-Making Part 2: Pricing Optimization and Revenue Management

```
def simulate_pricing_scenario(data, price_adjustment_pct):
        simulated = data.copy()
        simulated["Simulated_Price_USD"] =
simulated["Current_Price_USD"] * (1.0 + price_adjustment_pct)
        # Quantity adjusted by elasticity
        simulated["Simulated_Daily_Units"] =
simulated["Historical_Daily_Sales_Units"] * (1.0 +
(elasticity_coefficient * price_adjustment_pct))
        simulated["Simulated_Gross_Profit_USD"] =
(simulated["Simulated_Price_USD"] - simulated["Unit_Cost_USD"]) *
simulated["Simulated_Daily_Units"]
        return simulated["Simulated_Gross_Profit_USD"].sum()

    current_profit = simulate_pricing_scenario(df, 0.0)
    plus_5_profit = simulate_pricing_scenario(df, 0.05)
    minus_5_profit = simulate_pricing_scenario(df, -0.05)

    print(f"Current Daily Gross Profit: ${current_profit:,.2f}")
    print(f"Daily Gross Profit at +5% Price: ${plus_5_profit:,.2f}")
    print(f"Daily Gross Profit at -5% Price: ${minus_5_profit:,.2f}")
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 3: AI Applications in Business Decision-Making Part 2: Pricing Optimization and Revenue Management

```
# Days of inventory remaining at current sales velocity
    df["Days_Of_Inventory_Remaining"] = df["Inventory_Units_Count"] /
df["Historical_Daily_Sales_Units"]
    df["Markdown_Recommended"] =
df["Days_Of_Inventory_Remaining"].apply(lambda d: "Markdown_15Pct" if
d > 20 else "Maintain_Price")
    print(df[["Product_ID", "Inventory_Units_Count",
"Days_Of_Inventory_Remaining", "Markdown_Recommended"]].head(3))
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Exercise 1 and Exercise 2

EXERCISE 1: GUIDED

Foundational Practice

Follow the lab protocol, run the script, and verify its output with the session dataset.

EXERCISE 2: DEBUGGING

Applied Challenge

Extend the program, diagnose an intentional error, apply the verified correction, and confirm return code 0.